



**INSTITUT INTERNATIONAL
D'INGÉNIERIE
DE L'EAU ET DE L'ENVIRONNEMENT (2iE)**

Département Informatique, Intelligence Artificielle
et Applications (IIAA)

RAPPORT DE PROJET

Application de réservation de billets de transport

Bus, trains et vols au départ de Ouagadougou

Cours de Programmation Orientée Objet en Java
ECUE IIAA511, Semestre 5

RÉALISÉ PAR

SIONE Totta Yasser Soufiane
YELKUNI Tindwendé Franck Onel
NIKIEMA P. Hanifah
NADEMBEGA Ingrid Océane
Groupe 01

ENCADRÉ PAR

Dr Babacar LEYE

DATE

1er octobre 2026

TABLE DES MATIÈRES

Introduction générale	1
1 Analyse des besoins	2
1.1 Acteurs et périmètre	2
1.2 Besoins fonctionnels	2
1.3 Règles de gestion	3
1.4 Cycle de vie d'un ticket	3
2 Évolution du projet	4
2.1 La première version et ses limites	4
2.2 Les demandes de l'enseignant et nos réponses	4
2.3 Pourquoi Occupable	4
2.4 Ce que la version finale ajoute	5
2.5 Bilan chiffré de l'évolution	5
3 Architecture et conception	6
3.1 Cinq packages, une responsabilité chacun	6
3.2 Les classes principales	6
3.3 Les méthodes de l'agence	7
3.4 Nos principaux choix	7
3.5 Lire les diagrammes	8
4 Les notions du cours dans notre code	9
4.1 Interfaces, classe abstraite, héritage et polymorphisme	9
4.2 Généricité	9
4.3 Collections	9
4.4 Le contrat equals () et hashCode ()	10
4.5 Comparable, Comparator et méthodes utiles	10
4.6 Exceptions	10
4.7 Entrées, sorties et sérialisation	11
5 Tests et exécution du programme	12
5.1 Pourquoi tester, et à quels niveaux	12
5.2 Matrice des erreurs	12
5.3 Les 22 scénarios du programme de tests	13
5.4 Un test d'exception complet et le test du menu	14
5.5 Cas limites, résultats et corrections	15
5.6 Exécution de la démonstration	15
5.7 Utilisation du menu interactif	16
6 Organisation, difficultés et limites	17
6.1 Organisation du travail	17
6.2 Difficultés rencontrées	17
6.3 Limites et perspectives	18
Conclusion générale	19
Bibliographie	20
Annexe A Diagrammes de classes	21
Annexe B Schémas du cycle de vie et des couches	23
Annexe C Diagrammes de séquence	24

RÉSUMÉ

Ce rapport présente l'application de réservation de billets de transport que nous avons réalisée en Java dans le cadre du cours de Programmation Orientée Objet (ECUE IIAA511) du Département Informatique, Intelligence Artificielle et Applications de 2iE. Le programme simule le fonctionnement d'une petite agence qui vend des places de bus, de train et de vol au départ de Ouagadougou. Un voyageur choisit un trajet et réserve une place, ce qui crée un ticket. Il peut annuler ce ticket tant que le délai le permet : au moins 24 heures avant le départ pour un bus, 48 heures pour un train et 72 heures pour un vol. Quand le départ est atteint, les tickets encore réservés sont validés automatiquement et considérés comme payés. Quand un trajet est complet, le voyageur entre dans une file d'attente et reçoit la première place libérée par une annulation.

La première version du projet fonctionnait, mais elle utilisait peu les outils du cours. À la suite de la revue de notre enseignant, nous avons repris l'ensemble du code pour appliquer les interfaces, la classe abstraite, le polymorphisme, la généricité, les collections, le contrat `equals()` et `hashCode()`, les comparateurs, les exceptions personnalisées et les tests unitaires. L'interface `Reservable` est devenue `Occupable` et le code est réparti en cinq packages. La version finale ajoute le train, la sauvegarde de l'agence dans un fichier par sérialisation et un menu interactif en console.

Un principe a guidé tout le travail : une opération refusée ne doit laisser aucune trace. Toutes les vérifications précèdent donc toute modification, et chaque refus est signalé par une exception au nom explicite. Pour contrôler ce comportement, nous avons écrit 22 scénarios regroupant 182 vérifications et 109 tests unitaires JUnit répartis en huit classes. Le projet compte 23 classes et interfaces, environ 2 300 lignes de code et 1 100 lignes de tests, sans aucun commentaire dans le code.

Ce document décrit l'analyse des besoins, l'évolution du projet, l'architecture, l'application de chaque notion du cours, les tests, l'exécution du programme et l'organisation du travail. Les diagrammes sont regroupés en annexe.

Mots clés : Java, programmation orientée objet, interface, généricité, collections, exceptions, sérialisation, tests unitaires, JUnit.

ABSTRACT

This report presents the ticket booking application that we built in Java for the Object-Oriented Programming course (ECUE IIAA511) of the Department of Computer Science, Artificial Intelligence and Applications at 2iE. The program simulates a small travel agency that sells bus, train and flight seats from Ouagadougou. A traveller chooses a trip and books a seat, which creates a ticket. The traveller can cancel it as long as the deadline allows: at least 24 hours before departure for a bus, 48 hours for a train and 72 hours for a flight. At departure time, the tickets that are still booked are validated automatically and marked as paid. When a trip is full, the traveller joins a waiting queue and receives the first seat released by a cancellation.

The first version of the project worked, but it made little use of the tools studied in class. Following our teacher's review, we rebuilt the whole code base to apply interfaces, an abstract class, polymorphism, generics, collections, the equals and hashCode contract, comparators, custom exceptions and unit tests. The interface `Reservable` became `Occupable` and the code is organised into five packages. The final version adds trains, saving the agency to a file through serialization, and an interactive console menu.

One guiding principle drove the work: a refused operation must leave no trace. All checks therefore come before any modification, and every refusal is reported by an exception with an explicit name. To verify this behaviour we wrote 22 scenarios containing 182 checks and 109 JUnit unit tests split into eight classes. The project contains 23 classes and interfaces, about 2,300 lines of code and 1,100 lines of tests, with no comment in the code.

This document describes the requirements analysis, the evolution of the project, the architecture, the way each course concept is applied, the tests, the execution of the program and the organisation of the work. The diagrams are gathered in the appendices.

Keywords: Java, object-oriented programming, interface, generics, collections, exceptions, serialization, unit testing, JUnit.

LISTE DES TABLEAUX

Tableau 1 :	Besoins fonctionnels regroupés par thème	2
Tableau 2 :	Demandes de l’enseignant et réponses apportées	4
Tableau 3 :	Comparaison des trois étapes du projet	5
Tableau 4 :	Responsabilité des classes principales	6
Tableau 5 :	Méthodes publiques de la classe Agence	7
Tableau 6 :	Collections utilisées et raisons du choix	9
Tableau 7 :	Exceptions personnalisées et situations associées	10
Tableau 8 :	Classes de tests JUnit et contenu vérifié	12
Tableau 9 :	Matrice des erreurs : exception, effet du refus et tests associés	13
Tableau 10 :	Scénarios du programme de tests	13
Tableau 11 :	Répartition des tâches entre les membres du groupe	17
Tableau 12 :	Difficultés rencontrées et solutions retenues	17
Tableau 13 :	Limites de la version actuelle et améliorations envisagées	18

LISTE DES FIGURES

Figure 1 :	Diagramme de classes : modèle et contrats	21
Figure 2 :	Diagramme de classes : couche service	22
Figure 3 :	Diagramme de classes : hiérarchie des exceptions	22
Figure 4 :	Cycle de vie d'un ticket	23
Figure 5 :	Organisation du projet en couches	23
Figure 6 :	Séquence de la réservation d'une place	24
Figure 7 :	Séquence de l'annulation d'un ticket avec promotion depuis la file	25

INTRODUCTION GÉNÉRALE

Quand on achète un billet de bus, de train ou d'avion, on a rarement l'impression que quelque chose de compliqué se passe derrière l'écran. Pourtant, l'agence qui vend ce billet doit suivre en permanence plusieurs informations : combien de places restent sur chaque trajet, qui a réservé quoi, qui a annulé, et qui attend qu'une place se libère. Une erreur sur l'une d'elles se voit tout de suite. Un voyageur arrive avec un ticket pour un bus déjà plein, ou une place annulée n'est jamais remise en vente et le véhicule part à moitié vide.

C'est ce genre de situation que nous avons choisi de modéliser dans le cadre du cours de Programmation Orientée Objet en Java. Notre agence est petite et fictive. Elle propose des bus vers des villes du Burkina Faso comme Bobo-Dioulasso, Ouahigouya ou Fada N'Gourma, des trains vers Banfora et des vols vers Abidjan ou Marseille. Le sujet est concret et tout le monde en comprend les règles sans explication. Il se prête aussi très bien à la programmation orientée objet : les objets sont évidents (voyageur, trajet, ticket), un bus, un train et un vol ont des points communs mais aussi des comportements différents, et les situations d'erreur sont nombreuses.

La question à laquelle nous voulions répondre est la suivante : comment écrire un programme de réservation qui reste correct quand tout ne se passe pas comme prévu ? Coder le cas heureux est facile. Ce qui est difficile, c'est le trajet complet, le voyageur qui réserve deux fois, l'annulation trop tardive, l'email déjà utilisé, le numéro de trajet qui n'existe pas ou le fichier de sauvegarde illisible. Un programme fiable refuse ces cas proprement, avec un message clair, sans laisser la moindre trace de l'opération refusée.

Notre première version répondait déjà à une partie du problème, mais elle exploitait peu les outils du cours. À la suite de la revue de notre enseignant, nous l'avons reprise en entier avec trois objectifs : appliquer réellement les notions étudiées, séparer le modèle, la logique métier et l'interface, et vérifier chaque comportement par un test, en particulier les cas d'erreur. Pour cette version finale, nous sommes allés plus loin : nous avons ajouté le train pour montrer que l'architecture se prolonge sans casser l'existant, la sauvegarde de l'agence dans un fichier pour appliquer les entrées et sorties, et un menu interactif pour que le programme soit utilisable sans écrire de code.

Ce projet nous a demandé plus de réflexion que de lignes de code. Beaucoup de décisions ont porté sur ce qu'il ne fallait pas faire : ne pas laisser un ticket décider seul du type de trajet, ne pas signaler une erreur par un simple booléen, ne pas écrire à l'écran depuis le modèle, ne pas modifier l'état avant d'être certains que l'opération est valide. Chacune de ces décisions est expliquée dans ce rapport et reliée à une notion du cours.

Le rapport suit cette démarche. Le chapitre 1 présente l'analyse des besoins et les règles de gestion. Le chapitre 2 raconte l'évolution du projet. Le chapitre 3 explique l'architecture et nos choix de conception. Le chapitre 4 montre où chaque notion du cours est appliquée. Le chapitre 5 décrit les tests et l'exécution du programme. Le chapitre 6 présente l'organisation du travail, les difficultés et les limites. Une conclusion générale et la bibliographie précèdent les annexes, qui regroupent les diagrammes et les schémas.

CHAPITRE 1 : ANALYSE DES BESOINS

1.1 Acteurs et périmètre

Trois acteurs interviennent dans le programme : le **voyageur**, qui réserve, annule et consulte ses tickets, l'**agent de l'agence**, qui enregistre les voyageurs, ajoute les trajets, consulte les statistiques et sauvegarde l'agence, et le **système**, qui valide les tickets au départ et donne les places libérées aux voyageurs en attente.

Nous avons fixé ce que le programme fait et ce qu'il ne fait pas. Il gère les voyageurs, trois types de trajets, les tickets et les files d'attente. Il recherche, trie et compte, il sauvegarde et recharge l'agence, et il propose une démonstration et un menu interactif en console. Il ne gère pas de vraies dates : le temps restant avant le départ est un nombre d'heures donné à la création du trajet. Il n'offre ni paiement en ligne, ni choix de siège, ni interface graphique.

1.2 Besoins fonctionnels

Le cahier des charges détaille vingt besoins. Le tableau 1 en donne les grandes familles.

Tableau 1: Besoins fonctionnels regroupés par thème

Thème	Ce que le programme doit faire
Enregistrement	Enregistrer un voyageur (nom non vide, email valide, numéro et email uniques) et ajouter un trajet de bus, de train ou de vol (numéro unique, deux villes différentes, places et délai positifs ou nuls).
Réservation	Créer un ticket réservé et retirer une place. Refuser si le voyageur a déjà un ticket actif sur ce trajet.
File d'attente	Sur un trajet complet, placer le ticket en attente sans retirer de place, indiquer sa position, puis lui donner la première place libérée.
Annulation	Annuler un ticket réservé si le délai est respecté et rendre la place. Refuser sans rien modifier dans tous les autres cas.
Validation	Faire passer les tickets réservés d'un trajet à l'état payé au départ.
Consultation	Chercher les trajets disponibles ou par villes, lister les tickets d'un voyageur, trouver les voyageurs communs à deux trajets.
Statistiques	Compter les tickets par statut, calculer le chiffre d'affaires, lister les villes desservies, classer les voyageurs par nom.
Sauvegarde	Écrire l'agence complète dans un fichier et la recharger plus tard sans rien perdre.
Menu	Offrir un menu interactif qui donne accès à toutes ces fonctions et qui supporte les saisies invalides.
Erreurs	Signaler chaque erreur métier par une exception personnalisée au nom explicite.

1.3 Règles de gestion

Quinze règles encadrent le comportement du programme. Les plus structurantes sont les suivantes. Un bus peut être annulé si le départ est dans plus de 24 heures, un train si le départ est dans plus de 48 heures et un vol s'il est dans plus de 72 heures : à exactement 24 heures pour un bus, l'annulation est donc refusée. Le nombre de places d'un trajet n'est jamais négatif et le prix d'un ticket est positif ou nul, en francs CFA. Deux voyageurs ne peuvent pas partager le même email, même si les majuscules diffèrent. Un ticket annulé, payé ou refusé ne peut plus être annulé. La file d'attente respecte l'ordre d'arrivée. Enfin, une sauvegarde est d'abord écrite dans un fichier temporaire : si elle échoue, l'ancien fichier reste intact.

La règle la plus exigeante : une opération refusée ne modifie ni le statut d'un ticket, ni le nombre de places, ni les collections de l'agence, ni aucun fichier. Elle nous oblige à toujours vérifier avant de modifier, jamais l'inverse.

1.4 Cycle de vie d'un ticket

Un ticket possède cinq statuts : `RESERVE`, `PAYE`, `ANNULE`, `REFUSE` et `EN_ATTENTE`. Il naît `RESERVE` si le trajet a une place libre, et `REFUSE` s'il est complet. Quand un ticket refusé passe par l'agence, il devient `EN_ATTENTE`. Un ticket en attente devient `RESERVE` dès qu'une place se libère, ou `ANNULE` si le voyageur renonce. Un ticket réservé devient `PAYE` au départ, ou `ANNULE` si l'annulation est faite dans les délais. `PAYE` et `ANNULE` sont des états finaux. Le schéma correspondant est présenté à la figure 4, en annexe.

CHAPITRE 2 : ÉVOLUTION DU PROJET

2.1 La première version et ses limites

La première version contenait onze fichiers rangés dans un seul dossier : `Voyageur`, `Trajet`, `TrajetBus`, `TrajetVol`, `Ticket`, `Reservable`, `StatutTicket`, deux exceptions, `Main` et `TestsScenarios`. Elle gérait déjà les délais d'annulation, mais à la relecture plusieurs choses nous gênaient : aucune collection, aucune classe générique, aucun `equals()` redéfini et seulement deux exceptions personnalisées. Surtout, un défaut caché : le constructeur de `Ticket` ignorait le résultat de la réservation de place, ce qui laissait naître un ticket réservé sur un trajet complet.

2.2 Les demandes de l'enseignant et nos réponses

L'enseignant nous a demandé d'utiliser dans le projet les notions et les structures du cours, de séparer clairement le modèle, la logique métier et l'interface, de renommer l'interface `Reservable`, de soigner la gestion des exceptions et d'ajouter d'autres tests. Le tableau 2 résume notre réponse.

Tableau 2: Demandes de l'enseignant et réponses apportées

Demande	Notre réponse
Utiliser les notions et structures du cours	Ajout de <code>Depot<T></code> , <code>Agence</code> , des collections, de <code>Comparable</code> et <code>Comparator</code> , de <code>equals()</code> et <code>hashCode()</code> .
Séparer modèle, logique métier et interface	Cinq packages : contrats, modèle, exceptions, service et interface utilisateur.
Renommer l'interface <code>Reservable</code>	Elle s'appelle maintenant <code>Occupable</code> .
Gestion d'exceptions robuste	Passage de deux à sept exceptions, organisées en hiérarchie.
Ajouter d'autres tests	Passage de 8 scénarios à 22, plus 109 tests JUnit.
Perfectionner le projet	Train, sauvegarde par sérialisation, menu interactif, scripts de lancement.

2.3 Pourquoi `Occupable`

Le mot `Reservable` laissait entendre que l'interface décrivait quelque chose que l'on réserve. Or ce qu'elle décrit vraiment, c'est la capacité d'un trajet à compter ses places prises et libérées. Le nom `Occupable` colle mieux à cette idée et évite la confusion avec la méthode `reserver` de l'agence, qui fait bien plus que prendre une place : elle vérifie le voyageur, le trajet et le prix, puis gère la file d'attente. Les trois méthodes de l'interface, `reserverPlace()`, `libererPlace()` et `estComplet()`, parlent toutes de l'occupation du trajet.

2.4 Ce que la version finale ajoute

Le train. Un nouveau moyen de transport, avec un délai d'annulation de 48 heures, est ajouté par une simple classe fille de `Trajet`. Aucune ligne de `Ticket` ni d'`Agence` n'a changé pour cela : c'est la preuve concrète que l'architecture est ouverte à l'extension.

La sauvegarde. L'agence peut être écrite dans un fichier puis rechargée. Cette fonction applique les entrées et sorties et la sérialisation, évoquées dans le complément du cours, et introduit une septième exception, `SauvegardeException`.

Le menu interactif. Une classe `MenuConsole` permet d'utiliser toutes les fonctions du programme au clavier. Elle convertit les saisies avec contrôle, comme le fait l'exemple `NumberFormatException` du cours, et ne s'arrête jamais sur une saisie invalide.

Un modèle qui n'écrit plus à l'écran. Dans la version précédente, `Ticket` et `Agence` affichaient eux-mêmes certains messages. Nous avons retiré tous ces affichages : l'annulation renvoie maintenant la liste des tickets promus depuis la file, et la position dans la file est consultable par une méthode. Seules les classes du package d'interface écrivent à l'écran, ce qui répond pleinement à la demande de séparation entre logique et interface.

2.5 Bilan chiffré de l'évolution

Le tableau 3 compare les trois étapes du projet.

Tableau 3: Comparaison des trois étapes du projet

Critère	Première version	Après la revue	Version finale
Organisation du code	Un seul dossier	Cinq packages	Cinq packages
Interfaces	1	2	2
Classes génériques	0	1	1
Types de collections utilisés	0	6	6
Types de trajets	2	2	3
Exceptions personnalisées	2	6	7
Statuts d'un ticket	4	5	5
Sauvegarde dans un fichier	Non	Non	Oui
Menu interactif	Non	Non	Oui
Classes et interfaces du code	11	19	23
Scénarios de test	8	19	22
Vérifications des scénarios	non comptées	135	182
Tests JUnit	0	70	109

CHAPITRE 3 : ARCHITECTURE ET CONCEPTION

3.1 Cinq packages, une responsabilité chacun

Le code est rangé dans cinq packages. `billetterie.contrats` contient les interfaces, c'est-à-dire ce que les objets savent faire. `billetterie.modele` contient les données et leurs règles propres : `voyageur`, `trajets`, `ticket`, `statut`. `billetterie.exceptions` contient les erreurs métier. `billetterie.service` contient la logique de l'agence : réservations, recherches, statistiques, sauvegarde. `billetterie.ui` contient tout ce qui s'affiche à l'écran : la démonstration et le menu.

Les dépendances vont dans un seul sens : l'interface utilise le service, le service utilise le modèle, et le modèle s'appuie sur les contrats et les exceptions. Le modèle ne connaît ni le service ni l'affichage, ce qui permet de le tester tout seul. Le schéma des couches est présenté à la figure 5, en annexe.

3.2 Les classes principales

Le tableau 4 indique le rôle de chaque classe du projet.

Tableau 4: Responsabilité des classes principales

Classe	Responsabilité
Voyageur	Numéro, nom et email, immuables. Refuse un nom vide ou un email sans @.
Trajet	Classe abstraite : villes, date, heure, places. Réserve et libère les places. Déclare <code>delaiAnnulationHeures()</code> sans l'implémenter.
TrajetBus, TrajetVol, TrajetTrain	Fixent le délai d'annulation (24 h, 72 h et 48 h) et ajoutent la compagnie, le numéro de vol ou le numéro de train.
Ticket	Relie un voyageur à un trajet, porte un prix et un statut, gère ses propres transitions.
Depot<T>	Range des éléments par numéro et refuse les doublons.
Agence	Point d'entrée de la logique métier : réservations, file, recherches, statistiques.
Sauvegarde	Écrit une agence dans un fichier et la relit, en signalant toute erreur par une <code>SauvegardeException</code> .
Main	Démonstration complète, ou menu avec l'argument <code>menu</code> .
MenuConsole	Menu interactif : lit les saisies, appelle l'agence, affiche les résultats.

3.3 Les méthodes de l'agence

La classe `Agence` est le point d'entrée de la logique métier. Le tableau 5 présente ses méthodes publiques et la structure de données que chacune exploite.

Tableau 5: Méthodes publiques de la classe `Agence`

Méthode	Rôle	Structure utilisée
<code>enregistrerVoyageur</code>	Ajoute un voyageur et refuse un numéro ou un email déjà pris.	<code>Depot</code> , <code>HashSet</code>
<code>ajouterTrajet</code>	Ajoute un trajet de numéro unique.	<code>Depot</code>
<code>reserver</code>	Crée un ticket, ou le met en file si le trajet est complet.	<code>ArrayList</code> , <code>Queue</code>
<code>annuler</code>	Annule un ticket, donne la place au premier de la file et renvoie les tickets promus.	<code>Queue</code> , <code>List</code>
<code>positionDansFile</code>	Donne la position d'un ticket dans la file de son trajet.	<code>Queue</code>
<code>validerDepart</code>	Passe les tickets réservés d'un trajet à l'état payé.	<code>ArrayList</code>
<code>ticketsDuVoyageur</code>	Liste les tickets d'un voyageur.	<code>ArrayList</code>
<code>voyageursDuTrajet</code>	Donne les voyageurs qui ont une place.	<code>HashSet</code>
<code>voyageursCommuns</code>	Donne les voyageurs présents sur deux trajets.	<code>retainAll()</code>
<code>trajetsDisponibles</code>	Liste les trajets non complets, triés par date.	<code>Comparable</code>
<code>rechercherTrajets</code>	Cherche par villes, sans tenir compte de la casse.	<code>ArrayList</code>
<code>villesDesservies</code>	Donne les villes d'arrivée, triées et sans doublon.	<code>TreeSet</code>
<code>voyageursTries</code>	Classe les voyageurs avec le comparateur choisi.	<code>Comparator</code>
<code>compterParStatut</code>	Compte les tickets de chaque statut.	<code>TreeMap</code>
<code>chiffreAffaires</code>	Additionne le prix des tickets payés.	<code>ArrayList</code>
<code>tailleFileAttente</code>	Donne le nombre de tickets qui attendent sur un trajet.	<code>HashMap</code>

Note. Toute méthode qui reçoit un numéro inconnu lève une exception de type `ElementIntrouvableException`.

3.4 Nos principaux choix

Le ticket ne teste jamais le type du trajet. Pour savoir si un ticket peut être annulé, il compare le délai restant à `trajet.delaiAnnulationHeures()`, sans jamais demander si le trajet est un

bus, un train ou un vol. L'ajout du train l'a prouvé : il a suffi d'écrire une classe fille de `Trajet`. C'est le principe ouvert à l'extension et fermé à la modification.

Une exception plutôt qu'un booléen pour l'annulation. Un booléen peut être ignoré sans que personne ne s'en aperçoive. Une exception vérifiée oblige l'appelant à réagir, sinon le code ne compile pas.

Un ticket gère son propre échec. Le constructeur de `Ticket` demande une place au trajet. Si le trajet est complet, le ticket naît `REFUSE`, ce qui corrige le défaut de la première version. L'agence le transforme ensuite en `EN_ATTENTE` et le range dans la file du trajet.

Deux niveaux de validation. Les fautes de programmation (nom vide, prix négatif) lèvent une `IllegalArgumentException` non vérifiée. Les erreurs métier (trajet complet, doublon, fichier illisible) lèvent une exception personnalisée vérifiée. C'est la distinction du cours entre exceptions vérifiées et non vérifiées.

L'agence relie tout. Elle retrouve le voyageur et le trajet par leurs numéros, vérifie les règles, crée le ticket et le range. Un ticket n'a ainsi jamais besoin de connaître les autres tickets pour détecter un doublon.

Le modèle n'écrit jamais à l'écran. Les méthodes du modèle et du service renvoient leurs résultats, et c'est l'interface qui les affiche. Cela rend ces classes testables sans lire une sortie console et permet de changer d'interface sans les toucher : la démonstration et le menu utilisent exactement les mêmes méthodes.

Une sauvegarde qui ne détruit rien. L'agence est d'abord écrite dans un fichier temporaire, puis ce fichier est déplacé vers sa destination. Si l'écriture échoue en route, l'ancien fichier reste intact et le fichier temporaire est supprimé : c'est le même principe d'atomicité que pour les réservations.

3.5 Lire les diagrammes

Les diagrammes sont regroupés en annexe. Le diagramme de classes tient sur trois figures. La figure 1 montre le modèle et les contrats : les flèches pointillées à triangle vide relient `Voyageur`, `Ticket` et `Trajet` à l'interface `Affichable`, et `Trajet` à `Occupable` ; les flèches pleines à triangle vide relient `TrajetBus`, `TrajetVol` et `TrajetTrain` à leur classe mère abstraite, écrite en italique ; les flèches simples montrent qu'un ticket est associé à un voyageur, à un trajet et à un statut. La figure 2 montre la couche service : `Agence` contient deux `Depot`, une liste de tickets, un ensemble d'emails et une mappe de files d'attente ; elle est sérialisée par `Sauvegarde` et utilisée par `MenuConsole`. La figure 3 montre la hiérarchie des six exceptions métier. La figure 4 reprend les transitions du cycle de vie d'un ticket et la figure 5 les couches du projet. Enfin, les figures 6 et 7 sont des diagrammes de séquence : elles montrent, appel après appel, ce qui se passe quand on réserve et quand on annule.

CHAPITRE 4 : LES NOTIONS DU COURS DANS NOTRE CODE

4.1 Interfaces, classe abstraite, héritage et polymorphisme

Affichable déclare `afficher()`. Occupable déclare les trois opérations de gestion des places. Trajet implémente les deux, plus `Comparable<Trajet>`, alors qu'une classe ne peut hériter que d'une seule classe : c'est l'intérêt des interfaces mis en avant par le cours.

Extrait de code 4.1: Classe abstraite Trajet et sa classe fille TrajetBus

```
public abstract class Trajet implements Occupable, Affichable, Comparable<Trajet> {  
    public abstract int delaiAnnulationHeures();  
}  
public class TrajetBus extends Trajet {  
    @Override  
    public int delaiAnnulationHeures() { return 24; }  
}
```

Trajet ne peut pas être instanciée. Elle mélange des méthodes complètes (`reserverPlace()`, `libererPlace()`) et une méthode abstraite que chaque classe fille complète : 24 heures pour TrajetBus, 48 pour TrajetTrain et 72 pour TrajetVol. Le polymorphisme apparaît dans Ticket, où l'appel à `delaiAnnulationHeures()` exécute la bonne version selon le type réel du trajet, et dans les tests, où une seule boucle appelle `afficher()` sur un tableau d'Affichable.

4.2 Généricité

La classe `Depot<T>` reprend l'idée de la classe `Boite<T>` du cours. Le paramètre `T` est remplacé à l'utilisation.

Aucun transtypage n'est nécessaire et le compilateur refuse d'ajouter un trajet dans un dépôt de voyageurs. Sans la généricité, il aurait fallu écrire deux classes presque identiques, ou stocker des `Object` avec le risque de `ClassCastException`.

4.3 Collections

Le tableau 6 résume le choix de chaque collection.

Tableau 6: Collections utilisées et raisons du choix

Structure	Où	Pourquoi
ArrayList	Tickets, résultats de recherche	Ordre d'ajout conservé, parcours simple.
HashSet	Emails utilisés, voyageurs d'un trajet	Test d'appartenance rapide, aucun doublon.
TreeSet	Villes desservies	Aucun doublon et affichage trié.
TreeMap	Depot<T>, comptage par statut	Clés triées, liste toujours dans l'ordre des numéros.

Structure	Où	Pourquoi
HashMap	Files d'attente par trajet	Accès direct à la file d'un trajet.
Queue, LinkedList	File d'attente d'un trajet	Le premier arrivé est le premier servi.

La promotion des tickets de la file se fait dans la méthode privée `promouvoirEnAttente()`. Elle retire le premier ticket avec `poll()` tant que la file n'est pas vide et que le trajet a une place libre, et elle se termine toujours, car chaque tour retire un ticket de la file. Elle ne fait aucun affichage : elle renvoie la liste des tickets promus et laisse l'interface décider quoi dire.

4.4 Le contrat `equals()` et `hashCode()`

Le cours insiste sur ce point : sans redéfinition, deux objets aux mêmes attributs sont considérés comme différents par un `HashSet`. Nous avons redéfini les deux méthodes ensemble dans `Voyageur`, `Trajet` et `Ticket`. L'identité de chaque objet est son numéro, déclaré `final` pour ne jamais changer pendant que l'objet est dans un ensemble.

Extrait de code 4.2: Redéfinition de `equals()` et `hashCode()` dans `Voyageur`

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Voyageur)) return false;
    return numero == ((Voyageur) o).numero;
}
@Override
public int hashCode() { return Objects.hash(numero); }
```

4.5 Comparable, Comparator et méthodes utiles

`Trajet` est comparable : l'ordre naturel est la date, puis l'heure, puis le numéro, ce qui affiche les trajets disponibles du plus proche au plus lointain. `Voyageur` est comparable par numéro. Pour classer les voyageurs par nom, un comparateur séparé, `ComparateurVoyageurParNom`, donne un second ordre. Toutes nos boucles sont des boucles `for-each`. Pour trouver les voyageurs présents sur deux trajets, nous utilisons `retainAll()`, qui ne garde que les éléments communs à deux ensembles, ce qui ne fonctionne que grâce à `equals()` et `hashCode()`.

4.6 Exceptions

Les six exceptions métier héritent de `BilletterieException`, qui hérite d'`Exception`. Ce sont des exceptions vérifiées : le compilateur oblige à les traiter ou à les déclarer avec `throws`.

Tableau 7: Exceptions personnalisées et situations associées

Exception	Quand elle est levée
<code>TrajetCompletException</code>	Il ne reste plus de place sur le trajet.
<code>AnnulationImpossibleException</code>	Délai dépassé ou ticket non annulable.

Exception	Quand elle est levée
ReservationInvalideException	Double réservation ou prix négatif.
DoublonException	Numéro ou email déjà utilisé.
ElementIntrouvableException	Voyageur, trajet ou ticket inconnu.
SauvegardeException	Écriture ou lecture d'un fichier de sauvegarde impossible.

Grâce à la hiérarchie, un seul `catch (BilletterieException e)` intercepte toutes ces erreurs, tandis qu'un `catch` plus précis en cible une seule. Les méthodes de `Agence` laissent les exceptions remonter, comme dans l'exemple du cours, et ce sont `Main` et `MenuConsole` qui les capturent. Le bloc `finally` de `Main` supprime le fichier temporaire de la démonstration.

Atomicité. Une opération est atomique si elle réussit complètement ou laisse l'objet dans son état initial. Dans `Agence.reserver()`, toutes les vérifications précèdent la création du ticket. Dans `Ticket.annuler()`, les refus sont testés avant de changer le statut ou de rendre la place.

Extrait de code 4.3: Vérifications préalables dans `Agence.reserver()`

```
Voyageur voyageur = voyageurs.trouver(numeroVoyageur);
Trajet trajet = trajets.trouver(numeroTrajet);
if (prix < 0) {
    throw new ReservationInvalideException("Le prix ne peut pas etre negatif.");
}
if (aDejaUnTicketActif(voyageur, trajet)) {
    throw new ReservationInvalideException("Ticket actif deja present sur ce trajet.");
}
Ticket ticket = new Ticket(prochainNumeroTicket, voyageur, trajet, prix);
```

4.7 Entrées, sorties et sérialisation

Le complément du cours présente la sérialisation comme un prolongement facultatif. Nous l'avons appliquée pour sauvegarder l'agence. Les classes `Voyageur`, `Trajet`, `Ticket`, `Depot` et `Agence` implémentent `Serializable`. Ses champs sont déclarés avec des types concrets sérialisables (`ArrayList`, `HashSet`, `HashMap`, `LinkedList`).

Extrait de code 4.4: Sauvegarde d'une agence dans un fichier

```
Path destination = Paths.get(chemin);
Path temporaire = Paths.get(chemin + ".tmp");
try (ObjectOutputStream sortie = new ObjectOutputStream(new FileOutputStream(temporaire))) {
    sortie.writeObject(agence);
} catch (IOException e) {
    supprimerSansErreur(temporaire);
    throw new SauvegardeException("Sauvegarde impossible dans " + chemin);
}
Files.move(temporaire, destination, StandardCopyOption.REPLACE_EXISTING);
```

Le bloc `try` avec ressource ferme automatiquement le flux, comme le ferait un `finally`. Toute `IOException` est traduite en `SauvegardeException`, ce qui évite de faire remonter des détails techniques jusqu'au menu. Côté interface, le menu lit le clavier avec `Scanner` et traite `NumberFormatException` comme dans l'exemple du cours.

CHAPITRE 5 : TESTS ET EXÉCUTION DU PROGRAMME

5.1 Pourquoi tester, et à quels niveaux

Écrire du code n'est que la moitié du travail : il faut pouvoir vérifier, à chaque modification, que rien n'a été cassé. Le cours donne trois raisons d'automatiser les tests : la non-régression, la documentation vivante et la couverture des cas limites. Nous les avons ressenties : plusieurs fois, une modification de `Ticket` a fait échouer un test de `Agence` que nous n'aurions jamais relancé à la main.

Le programme de scénarios. `TestsScenarios` est un programme Java ordinaire, sans bibliothèque. Il regroupe 22 scénarios et 182 vérifications, chacune affichant `[OK]` ou `[ECHEC]`, avec un bilan final. Il fonctionne partout, même sans JUnit.

Les tests JUnit. Huit classes testent chaque classe séparément avec `@Test`, `assertEquals`, `assertTrue` et `assertThrows`. Une méthode `@BeforeEach` recrée un état neuf avant chaque test pour qu'ils ne s'influencent pas. Le tableau 8 détaille leur contenu.

Tableau 8: Classes de tests JUnit et contenu vérifié

Classe de test	Tests	Ce qui est vérifié
<code>VoyageurTest</code>	7	Validation, <code>equals()</code> , <code>hashCode()</code> , <code>compareTo()</code> , <code>HashSet</code> .
<code>TrajetTest</code>	13	Places, délais, validation, tri par date et heure.
<code>TrajetTrainTest</code>	6	Délai de 48 heures, numéro de train, places, comparaison des trois types.
<code>TicketTest</code>	18	Statuts, annulation, validation, attente, égalité.
<code>DepotTest</code>	7	Ajout, recherche, doublons, copie indépendante, ordre.
<code>AgenceTest</code>	30	Réservation, file, position, validation, statistiques, recherches.
<code>SauvegardeTest</code>	9	Aller-retour, file conservée, indépendance, fichier absent ou invalide.
<code>MenuConsoleTest</code>	19	Saisies valides et invalides, réservation, annulation, sauvegarde, chargement.
Total	109	

5.2 Matrice des erreurs

Le tableau 9 lie chaque erreur possible à une exception, à l'endroit où elle est levée, à son effet sur l'état du programme et aux tests qui la vérifient.

Tableau 9: Matrice des erreurs : exception, effet du refus et tests associés

Erreur	Exception	Levée dans	Effet du refus	Tests
Trajet complet	TrajetCompleet Exception	Trajet	Places restent à zéro	Sc. 2, 17
Annulation hors délai	Annulation Impossible Exception	Ticket	Statut et places inchangés	Sc. 4, 16, 19
Ticket non annulable	Annulation Impossible Exception	Ticket	Aucun changement	Sc. 5
Double réservation	Reservation Invalide Exception	Agence	Aucun ticket créé	Sc. 12, 13
Prix négatif	Reservation Invalide Exception	Agence	Aucun ticket créé	Sc. 7, 12
Numéro ou email existant	DoublonException	Agence, Depot	Collections inchangées	Sc. 8, 11
Numéro inconnu	Element Introuvable Exception	Depot, Agence	Aucun changement	Sc. 12, 16
Fichier absent ou invalide	Sauvegarde Exception	Sauvegarde	Agence en cours inchangée	Sc. 20, 21
Donnée invalide	IllegalArgument Exception	Constructeurs	Objet non créé	Sc. 7, 18

Note. Sc. : numéro du scénario dans le programme de tests (tableau 10).

5.3 Les 22 scénarios du programme de tests

Chaque scénario du tableau 10 regroupe plusieurs vérifications autour d'un même thème.

Tableau 10: Scénarios du programme de tests

N°	Scénario	Ce qu'il contrôle
0	Interfaces	Occupable et Affichable utilisées de façon polymorphe sur des bus, des vols et une agence.
1	Réservation acceptée	Ticket RESERVE et une place retirée.
2	Trajet complet	Ticket REFUSE et places inchangées.
3	Annulation acceptée	Ticket ANNULE et place rendue.
4	Annulation refusée	Exception, statut et places inchangés.
5	Opérations interdites	Annuler deux fois, valider un ticket annulé, annuler un refusé.

N°	Scénario	Ce qu'il contrôle
6	Quatre voyageurs	Trois réservés et un refusé, sur un bus puis sur un vol.
7	Cas limites	24 h exactement, 73 h, nom vide, email invalide, prix négatif.
8	Dépôt générique	Ajout, recherche, doublon, copie indépendante.
9	<code>equals()</code> et <code>hashCode()</code>	Doublons éliminés dans un <code>HashSet</code> .
10	Tris	Comparable sur les trajets, <code>Comparator</code> sur les voyageurs.
11	Enregistrement	Doublons de numéro et d'email, insensibles à la casse.
12	Réservation par l'agence	Voyageur ou trajet inconnu, double réservation, prix négatif.
13	File d'attente	Ordre d'arrivée, promotion, retrait d'un ticket en attente.
14	Validation et statistiques	Comptage par statut et chiffre d'affaires.
15	Ensembles et recherche	Voyageurs communs, villes triées, recherche sans casse.
16	Atomicité	Un refus ne laisse aucune trace.
17	Hierarchie des exceptions	Chaque exception est capturée comme <code>BilletterieException</code> .
18	Validation des trajets	Villes, délais, compagnie, numéro de vol, délais d'annulation.
19	Train et polymorphisme	Délais 24, 72 et 48 heures, limite de 48 h, numéro de train vide.
20	Sauvegarde	Aller-retour complet, file conservée, fichier absent, dossier inexistant, fichier invalide.
21	Menu interactif	Saisies valides et invalides, réservation, annulation, sauvegarde, chargement.

5.4 Un test d'exception complet et le test du menu

Le cours précise qu'un test d'exception doit contrôler le type de l'exception et l'absence d'effet de bord. C'est ce que fait l'extrait suivant.

Extrait de code 5.1: Test JUnit d'une annulation hors délai

```
@Test
void annulationHorsDelaiLeveUneExceptionSansEffet() {
    Ticket ticket = new Ticket(1, voyageur, vol, 95000);
    assertThrows(AnnulationImpossibleException.class, () -> ticket.annuler());
    assertEquals(StatutTicket.RESERVE, ticket.getStatut());
    assertEquals(1, vol.getPlacesDisponibles());
}
```

Tester un menu interactif semble difficile, car il attend des saisies au clavier. Nous avons résolu le problème en donnant à `MenuConsole` un flux d'entrée et un flux de sortie en paramètres. En production, ce sont `System.in` et `System.out`. Dans les tests, ce sont un `ByteArrayInputStream`, qui contient la suite des saisies, et un `ByteArrayOutputStream`, dont on relit le contenu pour vérifier les messages affichés.

5.5 Cas limites, résultats et corrections

Nous avons testé l'annulation d'un bus à exactement 24 heures (refusée), d'un train à 48 heures (refusée) et d'un vol à 73 heures (acceptée), un trajet à zéro place dès sa création, l'annulation du même ticket deux fois (une seule place rendue), l'annulation d'un ticket payé, annulé ou refusé, un email en majuscules déjà utilisé en minuscules, quatre voyageurs pour trois places, et l'annulation d'un ticket en file d'attente (aucune place libérée). Pour la sauvegarde, nous avons testé un fichier absent, un dossier inexistant (aucun fichier ne doit être créé) et un fichier qui n'est pas une sauvegarde.

Le programme de scénarios se termine par Bilan : 182 test(s) reussi(s), 0 echec(s). Les 109 tests JUnit utilisent l'API standard de JUnit 5 : il suffit d'ajouter la bibliothèque au projet comme l'explique le README, puis de lancer tous les tests du dossier test.

Les tests nous ont fait corriger deux choses. D'abord le défaut de la première version : un ticket pouvait naître réservé sur un trajet complet. Ensuite, un de nos premiers tests supposait qu'un trajet complet le restait après une annulation ; il a échoué et nous avons compris que le scénario était faux, pas le code, ce qui a donné un test utile de plus : un trajet redevient disponible quand une place est rendue.

5.6 Exécution de la démonstration

Depuis un terminal, dans le dossier du projet, on compile avec `javac -d out $(find src -name "*.java")`, puis on lance `java -cp out billetterie.ui.Main` pour la démonstration et `java -cp out billetterie.tests.TestsScenarios` pour les scénarios. Des scripts du dossier `scripts` font la même chose sous Linux, macOS et Windows. La démonstration enchaîne les étapes dans un ordre progressif : enregistrement de quatre voyageurs et de sept trajets, deux refus attendus, recherche, réservations et annulations sur un bus, un vol et un train, trois erreurs métier provoquées volontairement, files d'attente, voyageurs communs, bilan et enfin sauvegarde puis rechargement de l'agence.

Extrait de code 5.2: Extrait de la sortie de la démonstration

```
=== Annulation refusee (vol, depart dans moins de 72h) ===  
Annulation refusee : le delai d'annulation de ce trajet (72h avant depart) est depasse.  
=== Train 107 : annulation refusee (depart dans moins de 48h) ===  
Annulation refusee : le delai d'annulation de ce trajet (48h avant depart) est depasse.  
=== Une annulation libere une place pour le premier de la file ===  
Ticket #6 annule. Remboursement declenche.  
Le ticket #9 quitte la file d'attente : une place lui est attribuee.  
=== Sauvegarde et rechargement de l'agence ===  
Meme nombre de tickets apres rechargement : true  
Meme chiffre d'affaires apres rechargement : true
```

Le bilan final affiche sept tickets réservés, un payé, quatre annulés, un en attente et treize tickets au total, avec un chiffre d'affaires de 95 000 FCFA qui ne compte que les tickets payés. Le fait que `Main` traverse tous ces cas sans jamais s'arrêter sur une erreur non prévue est en soi un test de robustesse.

5.7 Utilisation du menu interactif

Avec l'argument `menu`, le programme charge quatre voyageurs et sept trajets, puis ouvre le menu. L'extrait suivant montre une session réelle : deux réservations qui remplissent un bus, une troisième qui entre en file d'attente, l'annulation d'un ticket qui libère une place pour le troisième, et une saisie invalide qui est signalée sans arrêter le menu.

Extrait de code 5.3: Session du menu interactif

```
Votre choix : 4
Numero du voyageur : 3
Numero du trajet : 101
Prix en FCFA : 5000
Ticket 3 - NIKIEMA P. Hanifah - trajet 101 (...) - 5 000 FCFA - statut : EN_ATTENTE
Le trajet est complet : position 1 dans la file d'attente.
Votre choix : 5
Numero du ticket : 1
Ticket 1 annule.
Le ticket 3 obtient la place liberee.
Votre choix : 4
Numero du voyageur : 1
Numero du trajet : abc
Saisie invalide : il faut saisir un nombre entier, et non "abc".
```

CHAPITRE 6 : ORGANISATION, DIFFICULTÉS ET LIMITES

6.1 Organisation du travail

Le tableau 11 présente la répartition des responsabilités entre les membres du groupe.

Tableau 11: Répartition des tâches entre les membres du groupe

Membre	Contribution
SIONE Totta Yasser Soufiane	Trajet, TrajetBus, TrajetVol, TrajetTrain, Occupable, Depot<T>.
YELKUNI Tindwendé Franck Onel	Ticket, Agence et la file d'attente.
NIKIEMA P. Hanifah	Voyageur, les exceptions, Sauvegarde et le diagramme de classes.
NADEMBEGA Ingrid Océane	Main, MenuConsole, TestsScenarios et les tests JUnit.

La rédaction des documents a été partagée et chaque membre a relu le travail des autres. Nous avons procédé en six étapes : relire la première version et lister les demandes, renommer l'interface et créer les packages, écrire les tests en même temps que les classes, ajouter le train, la sauvegarde et le menu, mettre à jour les documents, puis compiler et lancer tous les tests une dernière fois. Le code est écrit en Java avec IntelliJ IDEA, les tests avec JUnit 5, et les documents en L^AT_EX compilés avec pdflatex.

6.2 Difficultés rencontrées

Le tableau 12 rassemble les principales difficultés et la solution que nous avons retenue.

Tableau 12: Difficultés rencontrées et solutions retenues

Difficulté	Solution
Un HashSet qui gardait des doublons	Redéfinir equals () et hashCode () ensemble.
Exception ou booléen pour annuler	Exception vérifiée, qui oblige l'appelant à réagir.
Garder l'état intact après un refus	Vérifier avant de modifier, puis tester l'état avant et après.
Ticket sur un trajet complet	Statut EN_ATTENTE et file dans l'ordre d'arrivée.
Ordre d'affichage d'un HashSet	Copier dans une liste et trier avant d'afficher.
Des messages affichés par le modèle	Renvoyer les résultats et laisser l'interface les afficher.
Une sauvegarde interrompue	Écrire dans un fichier temporaire puis le déplacer.

Difficulté	Solution
Tester un menu qui attend le clavier	Passer le flux d'entrée et le flux de sortie en paramètres.
Code lisible sans commentaire	Des noms complets, comme <code>reessayerReservation()</code> ou <code>aDejaUnTicketActif()</code> .

6.3 Limites et perspectives

Notre solution a des limites que nous assumons. Le tableau 13 les associe à l'amélioration que nous envisagerions.

Tableau 13: Limites de la version actuelle et améliorations envisagées

Limite actuelle	Amélioration envisagée
La date et l'heure sont de simples textes, et le délai avant le départ est donné en heures.	Utiliser <code>LocalDateTime</code> pour calculer le vrai délai à partir de la date du trajet.
Le programme s'utilise en console seulement.	Ajouter une interface graphique qui réutilise les mêmes méthodes de l'agence.
Un ticket refusé créé directement avec <code>Ticket</code> n'est pas mis en file, seule l'agence gère la file.	Faire de l'agence l'unique point de création des tickets.
Aucun choix de siège n'est possible.	Ajouter une classe <code>Siège</code> et numérotter les places de chaque trajet.
La sauvegarde est un fichier binaire propre à Java.	Proposer aussi un export lisible, par exemple en texte séparé par des virgules.
Aucun paiement réel n'est géré.	Ajouter une classe de paiement derrière une interface, sans toucher à <code>Ticket</code> .

Ces limites ne remettent pas en cause l'architecture : la plupart des améliorations se font en ajoutant une classe ou en remplaçant une implémentation, sans modifier le reste du code, grâce à la séparation en packages et à l'usage des interfaces.

CONCLUSION GÉNÉRALE

Ce projet avait un objectif simple à énoncer et exigeant à tenir : écrire un programme de réservation qui reste fiable quand tout ne se passe pas comme prévu. Nous pensons l'avoir atteint en reprenant complètement notre première version. L'application gère aujourd'hui les voyageurs, trois types de trajets, les tickets, les annulations selon des délais différents, la validation au départ et une file d'attente pour les trajets complets. Elle sauvegarde et recharge l'agence dans un fichier, et elle se pilote depuis un menu interactif. Elle refuse proprement chaque situation invalide grâce à sept exceptions personnalisées, et chaque refus laisse le programme exactement dans l'état où il se trouvait.

Sur le plan technique, le projet applique de façon concrète l'ensemble des notions du cours. Les interfaces `Affichable` et `Occupable` décrivent ce que les objets savent faire. La classe abstraite `Trajet` factorise le comportement commun d'un bus, d'un train et d'un vol. La classe générique `Depot<T>` évite de dupliquer le code. Les collections sont choisies selon le besoin : listes pour l'ordre, ensembles pour l'unicité, mappes pour l'accès par clé, file pour l'ordre d'arrivée. Le contrat `equals()` et `hashCode()` rend les ensembles fiables, et les comparateurs donnent deux ordres de tri. La hiérarchie d'exceptions permet de traiter les erreurs par famille ou une par une. Les entrées, les sorties et la sérialisation prolongent le cours en donnant au programme une mémoire. La séparation en cinq packages rend chaque fichier court et permet de tester le modèle indépendamment de l'affichage.

La qualité du résultat repose sur les tests. Les 22 scénarios, qui regroupent 182 vérifications, et les 109 tests unitaires contrôlent tous les comportements prévus, y compris les cas limites et les erreurs. Chaque test d'exception vérifie le type de l'erreur et l'absence d'effet sur l'état, ce qui transforme notre principe directeur en vérification automatique. Ils nous ont aussi servi de filet de sécurité : l'ajout du train et le retrait des affichages du modèle se sont faits sans crainte.

Au delà du résultat, ce travail nous a appris plusieurs choses. Le contrat `equals()` et `hashCode()` n'est pas un détail : un ensemble qui garde des doublons est un vrai bug. Une exception vérifiée protège mieux qu'un booléen. Une bonne conception se reconnaît à la facilité avec laquelle on peut la faire évoluer : le train a demandé une classe, pas une réécriture. Enfin, travailler en groupe demande autant d'efforts que programmer : répartir les classes, se mettre d'accord sur les noms et relire le code des autres nous a rendus plus rigoureux.

Notre solution a des limites, notamment l'absence de vraie gestion du temps, d'interface graphique et de paiement. Elles sont décrites au chapitre 6, avec l'amélioration envisagée pour chacune. Nous retenons trois réflexes pour la suite de nos études : vérifier avant de modifier, choisir la structure de données qui correspond au besoin, et tester chaque comportement, y compris les erreurs.

Nous remercions notre enseignant, Dr Babacar LEYE, dont le cours et les remarques ont largement guidé ce travail.

BIBLIOGRAPHIE

- [1] LEYE Babacar, *Programmation Orientée Objet en Java*, support de cours de l'ECUE IIAA511, Semestre 5, Département Informatique, Intelligence Artificielle et Applications, 2iE, 6 septembre 2026.
- [2] Oracle, *Documentation officielle de l'API Java* : collections, Comparable, Comparator, Objects, Files et sérialisation.
- [3] JUnit Team, *Documentation officielle de JUnit 5* : annotations et assertions.

ANNEXE A : DIAGRAMMES DE CLASSES

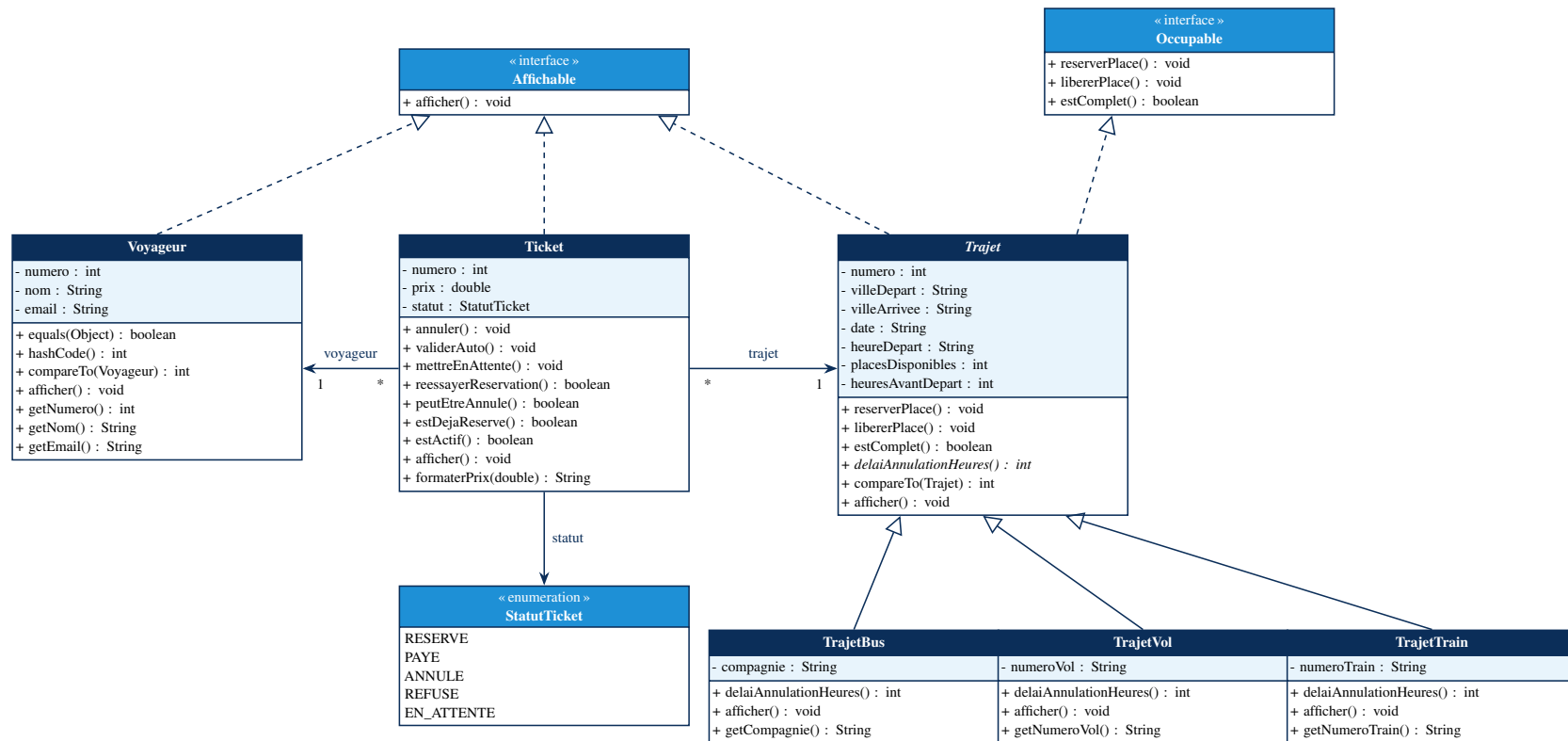


Figure 1: Diagramme de classes : modèle et contrats

Légende. Flèche pleine à triangle vide : héritage (extends). Flèche pointillée à triangle vide : implémentation (implements). Flèche simple : association. Nom en italique : élément abstrait. Cardinalités : 1 (un seul), * (plusieurs).

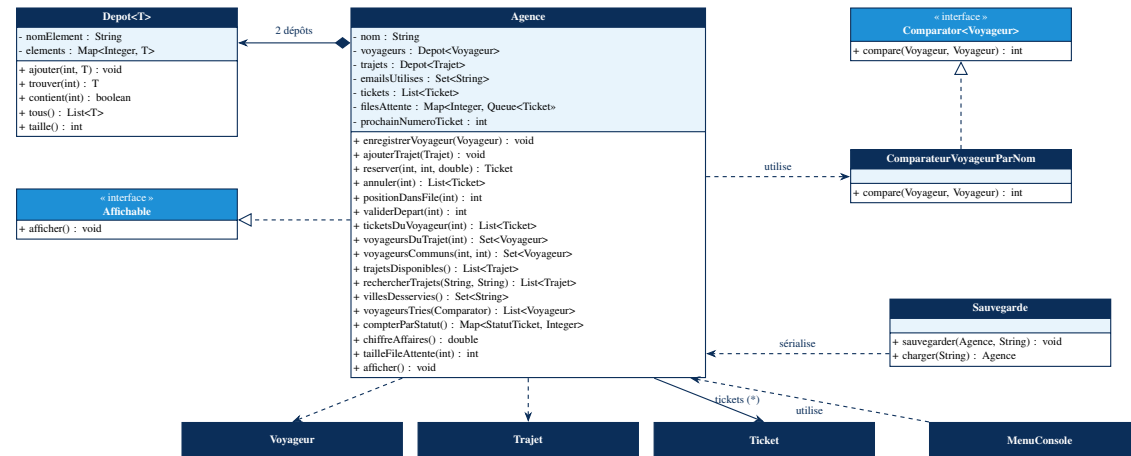


Figure 2: Diagramme de classes : couche service

Légende. Losange plein : composition. Flèche pointillée simple : dépendance. Classes du modèle rappelées en abrégé.

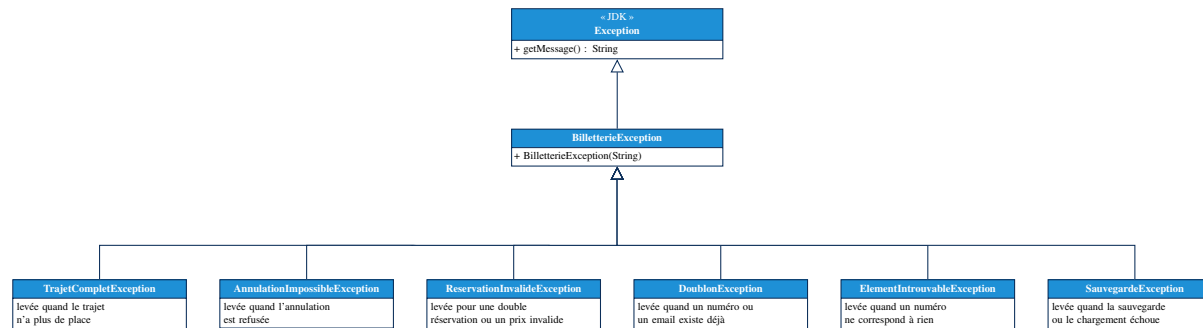


Figure 3: Diagramme de classes : hiérarchie des exceptions

Légende. Les six exceptions métier héritent de BilletterieException, elle-même fille de Exception (JDK). Toutes sont vérifiées.

ANNEXE B : SCHÉMAS DU CYCLE DE VIE ET DES COUCHES

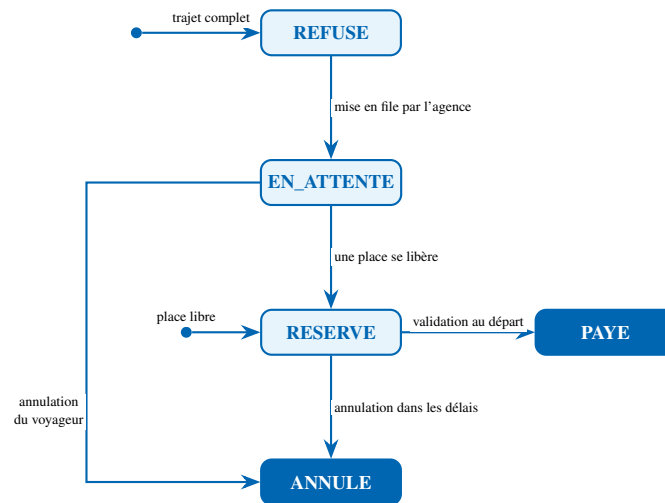


Figure 4: Cycle de vie d'un ticket

Légende. Rectangle clair : état intermédiaire. Rectangle plein : état final. Point plein : création du ticket. Un ticket créé sur un trajet complet naît **REFUSE** ; l'agence le place alors en attente.

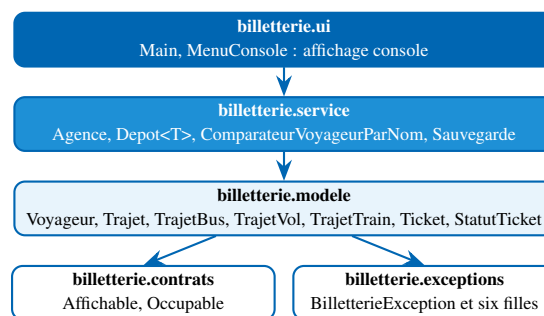


Figure 5: Organisation du projet en couches

Légende. Chaque flèche signifie « utilise ». L'interface dépend du service, le service dépend du modèle, et le modèle dépend des contrats et des exceptions. Aucune flèche ne remonte : le modèle ignore le service et l'affichage.

ANNEXE C : DIAGRAMMES DE SÉQUENCE

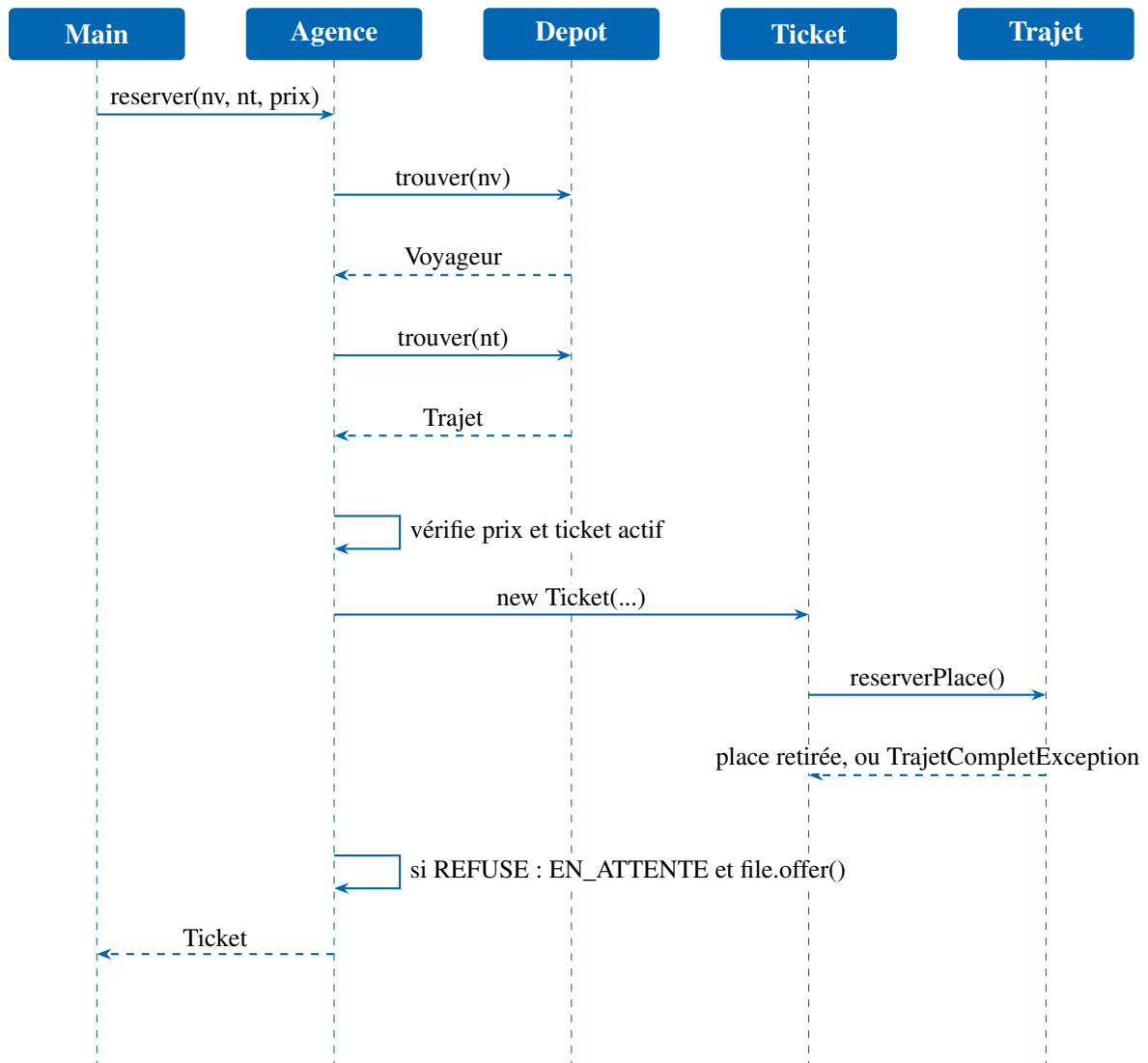


Figure 6: Séquence de la réservation d'une place

Légende. Flèche pleine : appel d'une méthode. Flèche pointillée : valeur renvoyée. Flèche qui revient sur la même ligne de vie : traitement interne de l'agence. Si le trajet est complet, le ticket devient REFUSE puis EN_ATTENTE et rejoint la file.

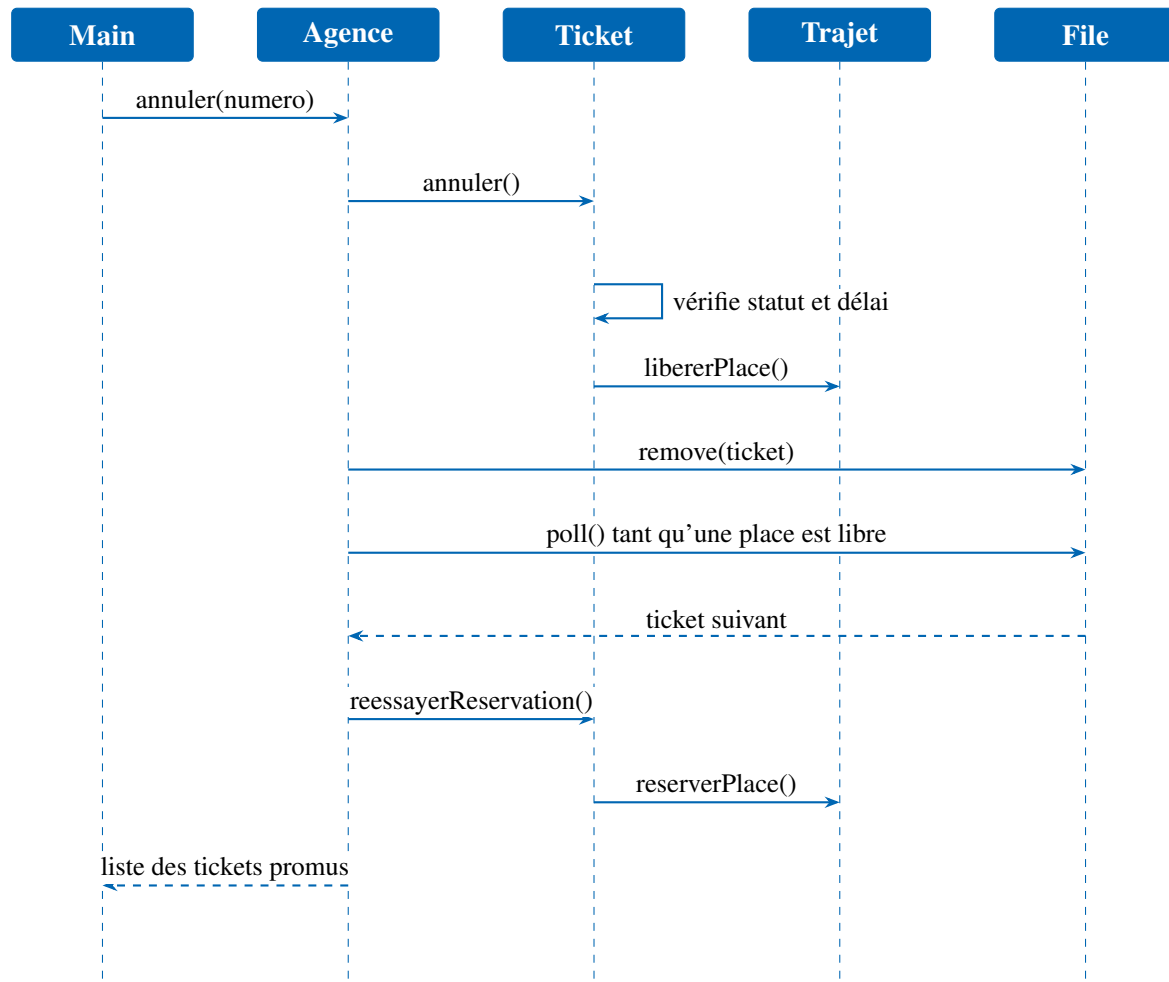


Figure 7: Séquence de l'annulation d'un ticket avec promotion depuis la file

Légende. Si le ticket est refusé (délai dépassé), l'exception est levée dès la vérification et rien d'autre ne se produit. Sinon, la place libérée est donnée au premier ticket de la file, et l'interface reçoit la liste des tickets promus pour l'afficher.